

UNIVERSIDADE PRESBITERIANA MACKENZIE

WELITON ANDRADE DA SILVA MARTINELI

**A EVOLUÇÃO DE JSP'S EM PROJETOS WEB JAVA: UM ESTUDO EM
PROJETOS DE CÓDIGO ABERTO.**

São Paulo

2015

WELITON ANDRADE DA SILVA MARTINELI

**A EVOLUÇÃO DE JSP'S EM PROJETOS WEB JAVA: UM ESTUDO EM
PROJETOS DE CÓDIGO ABERTO.**

Trabalho de Conclusão do Curso de Pós-graduação Lato Sensu apresentado à Universidade Presbiteriana Mackenzie para título de Especialização em Projetos e Desenvolvimento de Sistemas.

Orientador: Prof. Ms. Maurício Aniche

São Paulo

2015

MARTINELI, Weliton Andrade da Silva. A evolução de jsp's em projetos web java: Um estudo em projetos de código aberto. Trabalho de Conclusão de Curso (Especialização em Projetos e Desenvolvimento de Sistemas) – Centro de Tecnologia da Informação da Universidade Presbiteriana Mackenzie, São Paulo, 2015.

WELITON ANDRADE DA SILVA MARTINELI

**A EVOLUÇÃO DE JSP'S EM PROJETOS WEB JAVA: UM ESTUDO EM
PROJETOS DE CÓDIGO ABERTO.**

Trabalho de Conclusão do Curso de
Especialização em Projetos e Desenvolvimento
de Sistemas apresentado ao Centro de Tecnologia
da Informação da Universidade Presbiteriana
Mackenzie para obtenção do grau de Pós-
graduação Latu Sensu.

Aprovado em __ / __ / __

Prof. Ms. Maurício Aniche

Universidade Presbiteriana Mackenzie

Prof. Dr. Nome do Avaliador (avaliador)

Universidade de São Paulo

Dedico a minha esposa, Renata, pela paciência, compreensão e bondade com que sempre me apoiou na busca dos meus sonhos e a vocês, meus filhos Giovanna e Gustavo, por todos os sorrisos e abraços recebidos durante esse período.

AGRADECIMENTOS

Ao Prof. Ms. Maurício Aniche, muito obrigado, pelo apoio durante todo o trabalho, que com diretrizes seguras, muita paciência, constante acompanhamento e incentivo, me apoiou.

À minha família, pela paciência e compreensão.

Aos meus colegas de trabalho, pelas palavras de incentivo e apoio.

Aos meus colegas de sala, pela a companhia nesses anos de estudos.

“Você quer passar o resto da sua vida vendendo água com açúcar ou você quer uma chance de mudar o mundo? ” (Steve Jobs).

RESUMO

O *software* durante sua vida útil passa por um processo semelhante ao envelhecimento humano. Se o *software* é útil sua evolução torna-se essencial. Medir essa evolução é algo que não é fácil, porque estamos falando de uma grande quantidade de dados históricos gravados em controladores de versionamento de código. Uma análise desse tipo requer mecanismos sofisticados sob perspectivas e cálculos de métricas.

Neste documento apresentamos dados da evolução de JSPs, *Java Server Pages*, em projetos de código fonte aberto ao longo do tempo. A proposta é mostrarmos o que acontece com a qualidade do código nessa evolução, se eles pioram ou melhoram, se aparecem mais códigos *HTML*, mais *Scriptlets* ou *Taglibs*, quais as alterações que acontecem mais ao longo do tempo, entender se mudanças na camada de visualização impactam em outros módulos como no *controller* ou nos modelos.

A medição dessa evolução será feita utilizando técnicas de mineração de dados em repositórios de *software*. Com essa técnica, é possível analisar a evolução do *software* de forma automatizada aplicando técnicas de Mineração de Dados sobre o histórico do desenvolvimento de sistemas de *software*.

Com isso, é possível medir através de técnicas de mineração de dados, pontos a serem melhorados e ações premeditadas evitando o envelhecimento precoce do *software*.

Palavras-chave: Páginas-Servidor-Java.Mineração-Repositório-Software.Modelo-Visualização-Controle.

ABSTRACT

The software throughout its life goes through a process similar to human aging. If the software is useful evolution becomes essential. Measure this evolution is something that is not easy because we are talking about large amount of historical data recorded in versioning controllers code. Such an analysis requires sophisticated mechanisms under perspectives and metrics calculations.

In this paper we present data on the progress of JSPs, Java Server Pages, open source projects over time. The proposal is to show what happens to the quality of the code in this evolution, if they get worse or better, appear more HTML codes, or more scriptlets Taglibs, what changes happen more over time, understand whether changes in the view layer impact on other modules as the controller or the models.

The measurement of the revolution will be made using data mining techniques in software repositories. With this technique, it is possible to analyze the evolution of the automated software applying data mining techniques on historical development of software systems.

This makes it possible to measure using data mining techniques, points to be improved and premeditated actions preventing premature aging of the software.

Keywords: Java-Server-Pages. Mining-Software-Repositories. Model-View-Controller.

LISTA DE ILUSTRAÇÕES

GRÁFICO 1 HISTOGRAMA DE SCRIPTLET	37
GRÁFICO 2 BOXPLOT DE FREQUÊNCIA DE SCRIPTLET	37
GRÁFICO 3 HISTOGRAMA COMENTÁRIOS SCRIPTLETS	38
GRÁFICO 4 BOXPLOT FREQUÊNCIA COMENTÁRIOS SCRIPTLETS	38
GRÁFICO 5 HISTOGRAMA DE TAGLIB	39
GRÁFICO 6 BLOXPLOT FREQUÊNCIA TAGLIB	39
GRÁFICO 7 HISTOGRAMA HTML	40
GRÁFICO 8 BOXPLOT FREQUÊNCIA HTML.....	41

LISTA DE TABELAS

TABELA 1 DIRECTIVE (DIRETIVAS) - ELEMENTOS DE UMA JSP	15
TABELA 2 DECLARATION (DECLARAÇÕES) - ELEMENTOS DE UMA JSP	16
TABELA 3 SCRIPTLET - ELEMENTOS DE UMA JSP	16
TABELA 4 EXPRESSION (EXPRESSÕES) - ELEMENTOS DE UMA JSP	17
TABELA 5 ACTIONS (AÇÕES) - ELEMENTOS DE UMA JSP	17
TABELA 6 COMMENT (COMENTÁRIOS) - ELEMENTOS DE UMA JSP.....	18
TABELA 7 EXEMPLO MARTIN FOWLER.....	25
TABELA 8 EXEMPLO 1 DE JSP	26
TABELA 9 EXEMPLO 2 DE JSP	28
TABELA 10 EXEMPLO 3 DE JSP	29
TABELA 11 EXEMPLOS DE COMMITS (ENVIOS) AO REPOSITÓRIO GIT.....	32
TABELA 12 MÉTRICAS ANALISADAS	33
TABELA 13 LISTAGEM DOS PROJETOS ANALISADOS	35

LISTA DE ABREVIATURAS, SIGLAS E SÍMBOLOS

MVC	Modelo Visualização e Controle
JSP	Páginas de Servidores Java
MRS	Mineração de Repositório de Software
HTML	Linguagem de Marcação de Hipertexto

SUMÁRIO

1 MODEL-VIEW-CONTROLLER E A CAMADA DE VISUALIZAÇÃO COM JSPS.....	14
1.1 ELEMENTOS DE UM JSP	15
2 MINERAÇÃO DE DADOS	19
2.1 FERRAMENTA	22
3 DÍVIDA TÉCNICA.....	23
3.1 PROPRIEDADES DA DÍVIDA TÉCNICA.....	24
3.2 POSSÍVEIS DÍVIDAS TÉCNICAS EM JSP'S	26
4 METODOLOGIA.....	31
4.1 HEURÍSTICA	31
4.2 IMPLEMENTAÇÃO	32
5 CONSIDERAÇÕES FINAIS	42
REFERÊNCIAS.....	43
ANEXO.....	45

1 MODEL-VIEW-CONTROLLER E A CAMADA DE VISUALIZAÇÃO COM JSPS

Quando se fala em qualidade de software e divisão de responsabilidade, logo se pensa em Model-View-Controller. O MVC foi criado por Trygve Reenskaug em meados de 1978. Segundo Trygve, a primeira implementação do MVC e a documentação escrita foi realizada quando ele fazia uma visita científica ao Xerox Palo Alto Research Laboratory (PARC) nos anos de 1978-1979.

Segundo Glenn Krasner (1988), esse modelo de 3 camadas isola as unidades funcionais uma das outras, tanto quanto possível, isso torna mais fácil de compreender e modificar cada unidade, sem ter de saber tudo sobre outras unidades. Isso facilita a manutenção do software fazendo com que cada camada tenha sua própria responsabilidade.

O **Modelo (Model)** de aplicação é a simulação de software específico de domínio ou implementação da estrutura central do aplicativo, exemplo, a classe de domínio que representa uma entidade no banco de dados. A camada de **Visualização (View)** lida com todas as partes gráficas, ela solicita dados da camada de modelo e exibe os dados. Já a camada de **Controle (Controller)** é a interface entre a camada de modelo e a associada ao dispositivo de entrada a *View*. Os JSPs, que são o foco deste trabalho, ficam dentro da camada de visualização (a *View*) do MVC, é nessa camada que serão medidas a evolução dos JSPs ao longo do tempo.

JSPs em geral são páginas HTML com conteúdo dinâmico construídas utilizando código Java, *JavaBeans* e bibliotecas de *tags*, em última instância compiladas para *servlet*, elas recebem solicitações de clientes e enviam respostas de volta (MARK, 2003).

O JSP diferencia-se de conteúdos HTML pelos seus elementos. Os elementos JSPs são apresentados nas tabelas abaixo.

1.1 ELEMENTOS DE UM JSP

Tabela 1 Directive (Diretivas) - Elementos de uma Jsp

Tipo	Sintaxe	Interpretação
Directive (Diretivas)	<code><%@ ...%></code>	Diretivas fornecem informações gerais sobre a página JSP que será criada. Pode ser dividida em três categorias. (Diretivas de: <i>Include</i> , <i>Taglib</i> , <i>Page</i>).
Exemplo: <pre> <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %> <%@ taglib uri="http://www.example.com/custlib" prefix="mytag" %> <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <head> <title>Página Inicial</title> </head> <body> <%@include file="header.html" %> <hr/> <h2>Este é meu contexto principal</h2> <hr/> <mytag:hello/> <%@include file="footer.html" %> </body> </html> </pre>		

Tabela 2 Declaration (Declarações) - Elementos de uma Jsp

Tipo	Sintaxe	Interpretação
Declaration (Declarações)	<code><%! ...%></code>	A declaração é usada para definir variáveis e métodos na página JSP.
Exemplo: <pre> <html> <body> <%! int data=50; %> <%= "O valor da variável é: "+ data %> </body> </html> </pre>		

Tabela 3 Scriptlet - Elementos de uma Jsp

Tipo	Sintaxe	Interpretação
Scriptlet	<code><% ... %></code>	Scriptlets são códigos Java simples que podem ser utilizados na página JSP. Ele pode conter qualquer código Java. Normalmente usamos scriptlets embutidos em códigos HTML em páginas JSP. Ela começa com <code><%</code> e termina com <code>%></code> .
Exemplo: <pre> <html> <body> <% out.print("Bem Vindo ao Jsp"); %> </body> </html> </pre>		

Tabela 4 Expression (Expressões) - Elementos de uma Jsp

Tipo	Sintaxe	Interpretação
Expression (Expressões)	<code><%= ... %></code>	A expressão é usada para inserir o valor de uma expressão de linguagem de script, convertido em um texto, no fluxo de dados devolvido ao cliente.
Exemplo: <pre> <html> <body> <%= "Bem vindo ao Jsp" %> </body> </html> </pre>		

Tabela 5 Actions (Ações) - Elementos de uma Jsp

Tipo	Sintaxe	Interpretação
Actions (Ações)	<code><jsp: ... /></code>	Ações são os comandos enviados para o motor JSP para executar determinada tarefa.
Exemplo: <pre> <html> <body> <h2>Esta é minha página inicial</h2> <jsp:forward page="printdate.jsp" > <jsp:param name="name" value="javatpoint.com" /> </jsp:forward> </body> </html> </pre>		

Tabela 6 Comment (Comentários) - Elementos de uma Jsp

Tipos	Sintaxe	Interpretação
Comment (Comentários)	<code><%-- ... --%></code>	Comentários em páginas JSP, servem para comentar códigos que não serão mais utilizados ou para facilitar o entendimento de alguma lógica no código.
Exemplo: <pre> <html> <body> <%-- <%= "Bem Vindo ao Jsp" %> --%> </body> </html> </pre>		

Para analisar dados de elementos como esses em arquivos, é necessário que seja feito um levantamento utilizando técnicas de mineração de dados.

2 MINERAÇÃO DE DADOS

Durante a produção de software, garantir a qualidade do software consome um esforço considerável. Para aumentar a eficácia e eficiência desse esforço, é importante direcioná-los naqueles que mais precisam. Precisamos, portanto, identificar as peças de software que são as mais propensas a falhar e, portanto, exigem mais da nossa atenção. Uma fonte para determinar peças sujeitas a falhas pode ser o seu passado: se uma entidade de software (como um módulo, um arquivo, ou algum outro componente) teve probabilidade de falhar no passado, é provável que ocorra no futuro. Essa informação pode ser obtida a partir de bases de dados de bugs, especialmente quando combinada com informações de versões, de tal forma que se pode mapear as falhas de entidades específicas. No entanto, previsões precisas exigem histórias com longas falhas, que podem não existir para a entidade em questão. Nos últimos anos, os pesquisadores aprenderam a explorar a grande quantidade de dados contidas em repositórios de software, tais como versões e bancos de dados de bugs. A ideia principal é que se pode mapear problemas (no banco de dados de bugs) de correções (no banco de dados de versões) e assim, para esses locais no código que causaram o problema. Este mapeamento é a base de métricas que associam automaticamente com defeitos de pós-lançamento (NAGAPPAN, 2006).

Segundo Bing Liu 2007, a mineração de dados é também chamada de descoberta de conhecimento em bases de dados (*knowledge discovery in databases - KDD*). Ela é geralmente definida como um processo de descobrir padrões ou conhecimento de fontes de dados, por exemplo, bancos de dados, textos, imagens, a Web, etc. Os padrões devem ser válidos, potencialmente úteis e compreensíveis. A mineração de dados é um campo multidisciplinar que envolve a aprendizagem de máquina, estatísticas, bases de dados, inteligência artificial, recuperação de informação e visualização. Há muitas tarefas de mineração de dados. Alguns dos mais comuns são de aprendizagem supervisionada (ou classificação), a aprendizagem não supervisionada (ou agrupamento), mineração de regras de associação, e padrão de mineração sequencial. A aplicação de mineração de dados geralmente começa com um entendimento do domínio da aplicação por analistas de dados (mineiros de dados), que, em seguida, identificam fontes de dados adequados e os dados de destino. Com os dados, a mineração de dados pode ser realizada, o que é geralmente realizada em três principais etapas:

1. **Pré-processamento:** Os dados brutos geralmente não são adequados para a mineração devido a várias razões. Ele pode necessitar de ser limpo, de modo a remover os ruídos ou anomalias. Os dados também podem ser muito grandes e / ou envolvem muitos atributos irrelevantes, que exigem a redução de dados por meio de amostragem e seleção de atributos.
2. **A exploração de dados:** Os dados processados é então alimentado a um algoritmo de mineração de dados que irá produzir padrões ou conhecimento.
3. **Pós-processamento:** Em muitas aplicações, padrões nem todos descobertos são úteis. Esta etapa identifica aqueles úteis para aplicações. Várias técnicas de avaliação e visualização são usados para tomar a decisão.

Todo o processo (também chamado de processo de extração de dados) é quase sempre iterativo. Geralmente, leva muitas rodadas para alcançar resultados satisfatórios finais, que são depois incorporados em tarefas operacionais do mundo real. Mineração de dados tradicional utiliza dados estruturados armazenados em tabelas relacionais, planilhas ou arquivos simples na forma tabular. Mineração de dados vem tornando-se cada vez mais importante e popular.

Para realização dessa análise serão utilizadas técnicas de mineração de dados em repositórios de software. De acordo com (MENS, 2008), repositórios de *software*, tais como sistemas de controle de versão, sistemas de rastreamento de defeitos e a comunicação feita entre o pessoal de projetos são usados para ajudar a gerenciar o progresso de projetos de software.

Em particular, a análise dos dados históricos e de relatos de *bugs* ganhou importância porque eles armazenam informações valiosas para a análise da evolução do *software*, enquanto a recuperação dos dados que residem em sistemas de controle de versão como GIT ou *Subversion* tornou-se um tema bem explorado, o último desafio reside nos dados recuperados e sua interpretação (MENS, 2008).

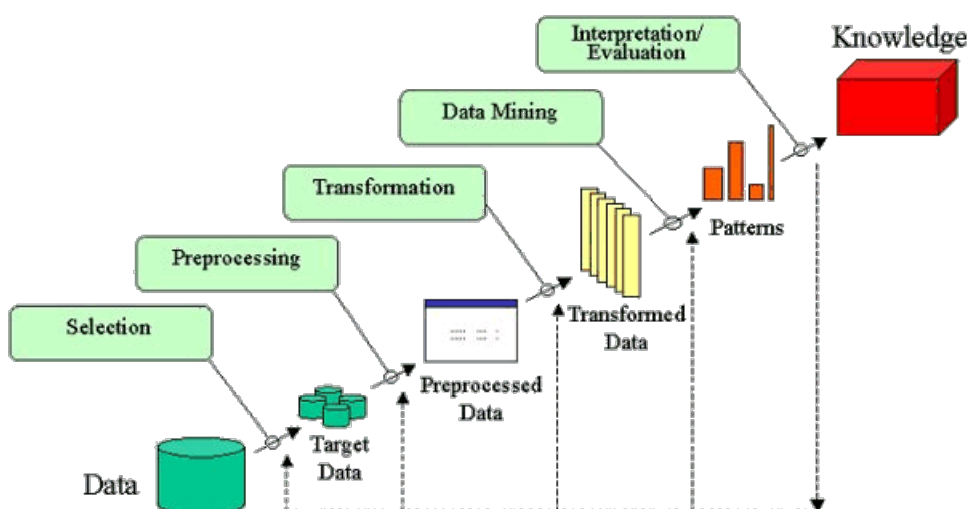
O primeiro passo de mineração consiste na criação de um modelo de dados de um sistema de *software* evoluído. Vários aspectos do sistema e sua evolução pode ser modelado: a última versão do código-fonte, o histórico de arquivos como registrado pelo sistema de controle de versão, várias versões do código-fonte (por exemplo, uma por *release*), documentação, relatórios de *bugs*, arquivos da lista de discussão de desenvolvedores, etc.

Ao projetar o modelo é importante considerar o balanço entre a quantidade de dados a tratar (na fase de análise) e o benefício potencial que estes dados podem ter, ou seja, nem todos aspectos da evolução de um sistema têm de ser considerados, mas apenas aqueles que podem resolver um problema de evolução do *software* específico ou um conjunto de problemas (MENS, 2008).

Uma vez que o modelo foi definido, um exemplo concreto tem que ser criado. Para isso, precisamos recuperar e processar a informação a partir das várias fontes de dados. O tratamento pode incluir a análise dos dados, a aplicação de correspondência (como exemplo, código fonte, arquivos, relatório de *bug* etc), técnicas para ligar diferentes fontes de dados. A análise consiste em utilizar os dados modelados e recuperados para resolver um problema de evolução do *software* ou conjunto de problemas por meio de diferentes técnicas e abordagens (MENS, 2008).

Quando nos referimos à história de um artefato de *software*, queremos dizer a forma como foi desenvolvido, como cresceu ou diminuiu ao longo do tempo, como muitos desenvolvedores trabalharam nele e a que medida. Esses tipos de informações são registradas pelos sistemas de controle de versão e podem ser recuperados analisando seus arquivos de log (MENS, 2008).

O processo global de encontrar e interpretar padrões de dados envolve a aplicação repetida das seguintes etapas:



Fonte: (FAYYAD, 1996)

Históricos de manutenções de longa duração podem ser sintomas de dívida técnica, visto que a manutenção é considerada a fase mais custosa da produção, equivalendo a cerca de 90% do custo total do ciclo de vida do software (LI, 2010).

Dívida técnica é uma nova forma de se concentrar na gestão a longo prazo de complexidades acidentais causadas por compromissos de curto prazo. Quando não se possui uma gestão correta de dívida técnica, acaba-se por incorrer em problemas a longo prazo como, por exemplo, custos com manutenção (BROWN, 2010).

A visão a respeito da metáfora dívida técnica vem ganhando força nas comunidades de desenvolvimento de software como um método inovador para compreender e difundir as questões de qualidade, custo e valor do software (OZKAYA, 2011).

No entanto, entender e gerir dívida técnica de modo a conseguir com que o projeto tenha a qualidade mínima necessária é uma tarefa difícil, que só parece possível através de uma combinação de fatores de diferentes áreas da engenharia de software como estudos qualitativos, métricas de software e planejamento (FALESSI, 2014).

2.1 FERRAMENTA

Para realização desse procedimento científico será utilizada a ferramenta MetricMiner, uma ferramenta web de apoio a mineração de repositórios de software. O MetricMiner surgiu a partir do rEvolution1, uma ferramenta de linha de comando que extrai dados de um repositório local e persiste em banco de dados relacional. Boa parte do código do rEvolution pôde ser reutilizada no MetricMiner, como o componente que realiza a interface com o sistema de controle de versão. Todo o código e o histórico de desenvolvimento do MetricMiner se encontra hospedado no github: <http://github.com/metricminer/metricminer>. (SOKOL).

3 DÍVIDA TÉCNICA

Ward Cunningham criou o termo dívida técnica como uma metáfora para o *trade-off* entre a escrita de código limpo em maior custo e atraso na entrega, e escrevendo código confuso barato e rápido à custa de esforços de manutenção mais elevados uma vez que é lançado em ambiente de produção. Joshua Kerievsky estendendo a metáfora à arquitetura e design, dívida técnica é semelhante à dívida financeira: ela suporta o desenvolvimento rápido no custo dos juros compostos a ser pago depois (BUSCHMANN, 2011).

Entregar sistemas de software cada vez mais complexos exige melhores maneiras de gerenciar os efeitos a longo prazo em expedientes de curto prazo. A metáfora dívida técnica está ganhando tração significativa na comunidade de desenvolvimento ágil como uma forma de compreender e comunicar tais questões. A idéia é que desenvolvedores, por vezes, aceitam compromissos em um sistema e em uma dimensão (por exemplo, a modularidade) para atender uma demanda urgente de alguma outra dimensão (por exemplo, um prazo), e que essas arbitragens acarretam em uma "dívida": aquela em que o "Juros" tem de ser pago e que o "Empréstimo" deve ser reembolsado em algum ponto a longo prazo para a saúde do projeto. (BROWN, 2010).

Uma maneira de entender dívida técnica é como uma maneira de caracterizar a lacuna entre o estado atual de um sistema de software e algumas hipótese de estado "ideal", em que o sistema é otimamente bem sucedido em um ambiente particular. Esta lacuna inclui itens que normalmente são monitorados em um projeto de software, como os conhecidos defeitos e características não implementadas. Mas também inclui menos aspectos óbvios e menos visíveis, como arquitetura e código em decadência e documentação desatualizada. Enquanto a metáfora é ampla o suficiente para abranger todos esses conceitos, o discurso em torno da dívida técnica tem enfatizado a última categoria, porque aquelas questões tendem a ser ignorados e desconsideradas pelos tomadores de decisão, quando considerando como investir em tempo de desenvolvimento. (BROWN, 2010).

3.1 PROPRIEDADES DA DÍVIDA TÉCNICA

(BROWN, 2010) também cita algumas propriedades de dívida técnica, são elas:

- **Visibilidade:** Problemas significativos surgem quando a dívida não é visível. Em muitos casos, é (ou era) conhecido por algumas pessoas (por exemplo, eu sei que eu quebrei encapsulamento para implementar um recurso antes do prazo), mas não é visível o suficiente para outras pessoas que eventualmente terão de pagar por isso. A finalidade da investigação neste área é encontrar formas de garantir que a dívida técnica alcance visibilidade adequada para que ele, o nível do sistema, possa ser considerado nos processos de tomada de decisão.
- **Valor:** No seu uso financeiro, a dívida quando gerenciada corretamente é um dispositivo de criação de valor (por exemplo, ter uma hipoteca permite possuir uma casa). O valor é a diferença econômica entre o sistema como ele é, e o sistema em um estado ideal para o ambiente assumido. Os atributos que permitem tal avaliação em software são difíceis de obter.
- **Valor presente:** Além do valor potencial do sistema global ativada por dívida técnica, o valor presente dos custos incorridos como resultado da dívida, incluindo o tempo do impacto e incerteza do impacto, deve ser mapeado para o análise global de custo-benefício.
- **Acréscimo da dívida:** Dívida não necessariamente é aditiva, mas super-aditiva no sentido de que, tendo muita dívida leva um sistema em um, talvez irreparável estado ruim (por exemplo, de complexidade de código).
- **Ambiente:** Em projetos de engenharia de software, a dívida é relativa a um determinado ambiente ou assumido.
- **Origem da dívida:** É importante distinguir claramente entre a dívida estratégica, tomada por alguma vantagem, e da dívida não intencional, que é tomado em qualquer meio de práticas pobres ou simplesmente porque o ambiente mudou na distância que criou uma incompatibilidade que reduz o valor do sistema.

- Impacto da dívida: A localidade (ou falta dela) da dívida é importante: são os elementos que precisam ser alterados para reembolsar uma dívida localizada ou muito espalhada.

Martin Fowler, categoriza dívida técnica em tipos distintos, separando as questões decorrentes da imprudência daquelas decisões que são feitas estrategicamente.

Tabela 7 Exemplo Martin Fowler

	Imprudente	Prudente
Intencional	"Não temos tempo para o design"	"Nós devemos enviar hoje e lidar com as consequências"
Sem Intenção	"O que é documentação?"	"Agora nós sabemos como deveria ter sido feito"

3.2 POSSÍVEIS DÍVIDAS TÉCNICAS EM JSP'S

Abaixo temos alguns exemplos de dívidas técnicas em códigos JSPs. No exemplo abaixo o desenvolvedor mesclou código java com html na camada de visualização utilizando scriptlets.

Tabela 8 Exemplo 1 de JSP

```
<%@page import="java.awt.FontMetrics" %>
<%@page import="java.awt.Graphics" %>
<%@page import="java.awt.image.BufferedImage" %>
<%@page import="java.io.*" %>
<%@page import="javax.servlet.http.*" %>
<%@page import="javax.servlet.*" %>
<%
    int rWidth = 200; //default width
    int rHeight = 200; //default height
    int FONT_HEIGHT = 10; //default fontsize
    String FONT_TYPE = "Arial"; //default font
    float QUALITY = 1f; //default image quality
    String rBackground = "#CCCCCC"; //default background
    String rColor = "#000000"; //default font color

    ServletOutputStream myout = response.getOutputStream(); //the output stream
    String[] params = request.getParameter("img").split("/"); //the parameters
    String paramText = request.getParameter("text"); //the text in your image

    String rText = "";

    for(int cp = 1;cp<params.length;cp++) {
        String current = params[cp];
```

```

        if (current!=null && current.trim().length()>0) {
            if (cp==1) {
                if (current.equalsIgnoreCase("wvga")) {
                    rWidth = 800; rHeight = 480;
                } else if (current.equalsIgnoreCase("svga")) {
                    rWidth = 800; rHeight = 480;
                } else if (current.equalsIgnoreCase("wxga")) {
                    rWidth = 1280; rHeight = 800;
                } else if (current.equalsIgnoreCase("wsxga")) {
                    rWidth = 1440; rHeight = 900;
                }
            }
            ...
        }

        //System.out.println("\tcp: "+current);
    }

    rText = rWidth+" x " +rHeight;

    if (paramText!=null&&paramText.trim().length()>0) {
        rText = paramText;
    }

    //process the image

    JPEGImageEncoder img = JPEGCodec.createJPEGEncoder(myout);
    JPEGEncodeParam param = img.getDefaultJPEGEncodeParam(image);
    param.setQuality(QUALITY, false);
    response.setContentType("image/jpg");
    img.encode(image,param);

%>

```

Já neste outro exemplo o desenvolvedor acessa uma classe java que faz parte da camada de modelo dentro da camada de visualização.

Tabela 9 Exemplo 2 de JSP

```
<%@pageimport="java.util.Iterator"%>
<%@pageimport="java.util.List"%>
<%@pageimport="br.edu.unidavi.bsi.daniel.model.Cliente"%>
<%@pageimport="br.edu.unidavi.bsi.daniel.jdbc.ClienteDAO"%>

<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Lista de Cliente</title>
...
</head>
<body>
<h1>Listagem de Clientes</h1>
<%
    ClienteDAO cDao = ClienteDAO.getInstance();
    List<Cliente> lista = cDao.getAll();
%>

<table border="1" class="linhasAlternadas">
  <tr class="even">
    <td><b>Nome</b></td>
    <td><b>Cidade</b></td>
    <td><b>UF</b></td>
    <td><b>Rua</b></td>
    <td><b>CEP</b></td>
    <td><b>Telefone</b></td>
    -- <td><b>Excluir</b></td>
  </tr>

<%
    Iterator<Cliente> iterator = lista.iterator();
    while (iterator.hasNext()) {
        Cliente c = iterator.next();
    %>
  <tr class="even">
    <td><%=c.getNome()%></td>
    <td><%=c.getEnderecoCidade()%></td>
    <td><%=c.getEnderecoEstado()%></td>
    <td><%=c.getEnderecoRua()%></td>
    <td><%=c.getEnderecoCep()%></td>
```

```

<td><%=c.getTelefone()%></td>
<td><a href='getExcluir.jsp?ID=<%= c.getId() %>'>Excluir</td>

<!--<td><img src='<%=request.getContextPath()%>/daniel/images/excluir.gif'
alt="Excluir" /></td>-->
<!--<td><input type='image'
src='<%=request.getContextPath()%>/daniel/images/excluir.gif'> </td>-->
<!--<td><img src='<%=request.getContextPath()%>/daniel/images/excluir.gif'
alt="Excluir" onclick="excluir()"/></td> -->
</tr>
<%
}
%>
</table>
</body>
</html>

```

Fonte: <https://github.com/>

No exemplo abaixo o desenvolvedor mescla bastante scriptlets com código java e acessa métodos de classe java diretamente pelo jsp

Tabela 10 Exemplo 3 de JSP

```

<%@page import="javax.swing.JOptionPane"%>
<%@page import="users.UserBean"%>
<%@page import="users.Pom"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<jsp:useBean id="user" scope="request" class="users.UserBean"/>
<jsp:useBean id="users" scope="application" class="users.UserDataBean"/>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title>

  </head>

  <body>
    <%--<jsp:setProperty name="user" property="*" />--%>
    <%
      //blok 1:
      user.setName(request.getParameter("name"));
      user.setSurname(request.getParameter("surname"));
    %>

```

```

user.setBirthYear(request.getParameter("birthYear"));

if (user.isValidData()) {
    //blok 2: ochrana pred F5: zapise to vzdy do poslednej beany,
    //pri F5 sa to len prepise.
    user = (UserBean) users.getLastBean();
    user.setName(request.getParameter("name"));
    user.setSurname(request.getParameter("surname"));
    user.setBirthYear(request.getParameter("birthYear"));
    %>
    <table border="4">
    <td> <%= "*" ID "*" %> </td> <td> <%= "*" JMÉNO "*" %> </td>
    <td> <%= "*" PŘIJMĚNÍ "*" %> </td> <td> <%= "*" ROK NAROZENÍ "*" %> </td>
    <%
    int rows = users.getSize();
    int cols = 4;

    for (int r = 0; r < rows; r++) {
        int id = users.getUsers().get(r).getIdNum();
        String name = users.getUsers().get(r).getName();
        String surname = users.getUsers().get(r).getSurname();
        String birthY = users.getUsers().get(r).getBirthYear();
        %>
        <tr>
        <% for (int c = 0; c < cols; c++) {
        %>
        <td> <%= id %> </td> <td> <%= name %> </td> <td> <%= surname %> </td>
        <td> <%= birthY %> </td>

        <div>
        <form action = "registerUser.jsp" method = "post">
        <input type="submit" value="Zpět" />
        </form>
        </div>
        </body>
    </html>

```

Fonte: <https://github.com/>

4 METODOLOGIA

Com base na discussão e motivações dado acima, este trabalho tem como objetivo responder a seguinte questão de pesquisa:

Qual é o padrão evolutivo do JSP ao longo do tempo ?

Para encontrar uma solução para esta questão, nós utilizamos métodos de estatística descritiva para organizar, resumir e descrever os aspectos importantes de um conjunto de características observadas ou comparar tais características entre três conjuntos *taglibs* (quando falamos taglib, levamos em consideração também as diretivas include e page), **scriptlets**, **comentários** **scriptlets** e *html*.

Algumas medidas que são normalmente usadas para descrever um conjunto de dados são medidas de tendência central e medidas de variabilidade.

Medidas de tendência central incluem **média**, **mediana** e **moda**. Medidas de variabilidade incluem **desvio padrão**, **variância**, **o valor máximo e mínimo**.

4.1 HEURÍSTICA

A heurística foi baseada em 23 projetos onde seriam analisados todos os *commits* seguindo as devidas condições:

Se encontrado no arquivo do tipo JSP o comando "<%", será iniciada a contagem de linhas scriptlet finalizando a contagem quando for encontrado o seguinte finalizador de scriptlet "%>". Para essa métrica não é considerado o comentário em scriptlets que inicia-se em <%-- e finalizam-se em --%>.

Se encontrado no arquivo do tipo JSP o comando "<%@", será iniciada a contagem de linhas de diretiva taglib, finalizando a contagem quando for encontrado o seguinte finalizador "%>".

Se encontrado no arquivo do tipo JSP o comando "<%--", será iniciada a contagem de linhas de comentários scriptlets, finalizando a contagem quando for encontrado o seguinte finalizador "--%>".

Se forem encontradas linhas no arquivo do tipo JSP, que não estejam contidas dentro do início e fim dos elementos citados acima, as linhas identificadas serão consideradas linhas HTML.

A heurística pode falhar em alguns casos, pois o *parse* (analisador) completo de um JSP pode ser complicado, mas, essas falhas entrarão no nosso desvio padrão.

4.2 IMPLEMENTAÇÃO

A ferramenta MetricMiner possibilitou a leitura de arquivos do tipo JSPs em repositórios de códigos fontes abertos. Esses arquivos são adicionados ou alterados a esses repositórios através de *commits* (envios) realizados pelos desenvolvedores de software. Cada mudança ou commit feito por um desenvolvedor gera um arquivo histórico de manutenção.

Exemplo:

Tabela 11 Exemplos de commits (envios) ao repositório GIT

Data	Arquivo	Descritivo	Desenvolvedor
01-01-2001	autenticacao.jsp (+)	Criação do arquivo	Denis
22-04-2005	autenticacao.jsp (+)	Adição da funcionalidade de palavra secreta	Maycon
14-08-2014	autenticacao.jsp (+)	Adição do captcha	Frederico

(+) Conteúdo do arquivo jsp

Fonte: Autor

O MetricMiner nos possibilitou configurar os projetos que seriam analisados. A ferramenta MetricMiner minerou todos os projetos setados, carregando essas informações em um modelo relacional facilitando o levantamento. Desta forma, todos os commits realizados dentre os projetos eram armazenados nesse modelo relacional separados por arquivos em etapas de commits. Com essas informações na base, foi possível gerar métodos em classes Java que lessem o conteúdo desses "arquivos" linha a linha identificando os elementos de uma JSP.

Elementos da JSP:

Tabela 12 Métricas Analisadas

Comando de início do elemento JSP	Comando de fim do elemento JSP	Descritivo do elemento JSP
<%@	%>	Diretivas (Taglib)
<%	%>	Scriptlets
<%--	--%>	Comentários Scriptlets
Qualquer outro comando	Qualquer outro comando	Html

O algoritmo implementado em java, passa por todas as linhas do arquivo em análise identificando os elementos da JSP, ao identificar o símbolo <%@ o algoritmo começa a contabilizar as linhas, ele termina a contagem quando identifica o símbolo %>, esse algoritmo continua com esse comportamento até o final do arquivo, contabilizando as demais diretivas de taglib encontradas.

O algoritmo de identificação de elementos scriptlets segue o mesmo padrão do anterior, mas, levando em consideração o elemento <% sem o símbolo (**arroba @**), para iniciar a contagem de linhas de scriptlets e finaliza ao encontrar o símbolo de scriptlets %>. Para esse caso também desconsideramos a expressão que inicia com <%= e finaliza com %>, ela serve para imprimir as saídas de valores scriptlets, fizemos isso para evitar duplicidade na contagem

de scriptlets. Também são desconsiderados comentários em scriptlets que inicia com `<%--` e finaliza com `--%>`.

O algoritmo faz também a checagem de elementos do tipo comentários em scriptlets tratando os elementos que iniciam com `<%--` e finalizam com `--%>`.

Para identificação dos elementos HTML, foram contabilizadas todas as linhas do arquivo em análise, subtraindo a quantidade de linhas encontradas dos elementos de diretiva taglib e scriptlets e comentários.

Resumindo, método Java percorria linha a linha de cada "arquivo" identificando o início e fim de cada elemento, contando a quantidade de linhas encontradas dentro desses elementos. As outras linhas que não faziam parte dos elementos eram consideradas comandos HTML. Em cada linha gerada de resultado obtinham-se as seguintes informações:

- Data de alteração;
- Diretório e nomenclatura completa do arquivo;
- Quantidade de linhas scriptlets encontradas;
- Quantidade de linhas de comentários scriptlets;
- Quantidade de linhas taglibs encontradas;
- Quantidade de linhas html encontradas.

Tabela 13 Listagem dos Projetos Analisados

Listagem dos Projetos Analisados
https://github.com/gxa/atlas
https://github.com/WASP-System/central
https://github.com/DonorConnect/Donor-Connect
https://github.com/INCF/eeg-database
https://github.com/tedeling/ehour
https://github.com/esporex-tv/esporex
https://github.com/dadastream/ff-core
https://github.com/TeamAwesome/forum
https://github.com/gitblit/gitblit
https://github.com/Graylog2/graylog2-web-interface
https://github.com/mifos/head
https://github.com/motech/motech-whp
https://github.com/php-coder/mystamps
https://github.com/OpenLMIS/open-lmis
https://github.com/OpenMRS-Australia/openmrs-cpm
https://github.com/QiBud/org.qibud.project
https://github.com/mpi2/PhenotypeArchive
https://github.com/vFabric/springtrader
https://github.com/motech/TAMA-Web
https://github.com/ostewart/toolkit

https://github.com/pykih/TracksAnalytics
https://github.com/crc83/WebCash
https://github.com/zanata/zanata-server

Fonte: Autor

O estudo realizado explorou quatro métricas, quantidade de *Scriptlets*, *Comment*, *Taglib* e *Html*. Dentre os projetos analisados foram encontrados cerca de 32.141 commits em arquivos jsp's, com isso foi possível interpretar os dados e entender que a quantidade de scriptlets tendem a diminuir com o passar do tempo. Isso é muito bom, porque foi possível perceber uma diminuição de dívida técnica nessa métrica.

Logo abaixo, é possível visualizar um resumo dos dados analisados em planilha. A tabela apresenta a quantidade máxima, primeiro quartil, mediana, terceiro quartil e mínimo encontradas de linhas em arquivos JSPs.

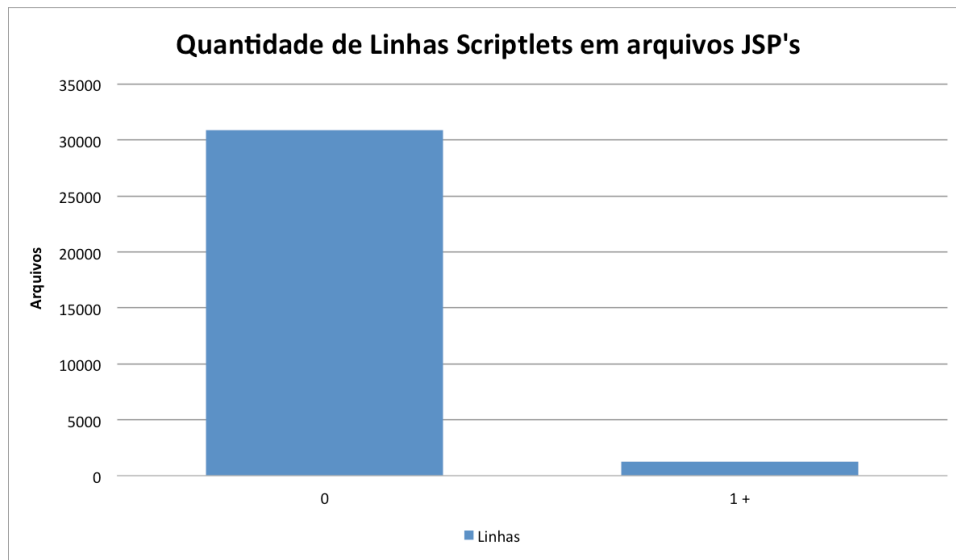
Tabela 13: Resumo de dados Analisados em Planilha

Funções	Scriptlets	Taglib	Html	Comentários
Máximo	125	18	1511	391
Primeiro Quartil	0	2	55	0
Mediana	0	5	122	6
Terceiro Quartil	0	8	244	19
Mínimo	0	0	0	0

Fonte: Autor

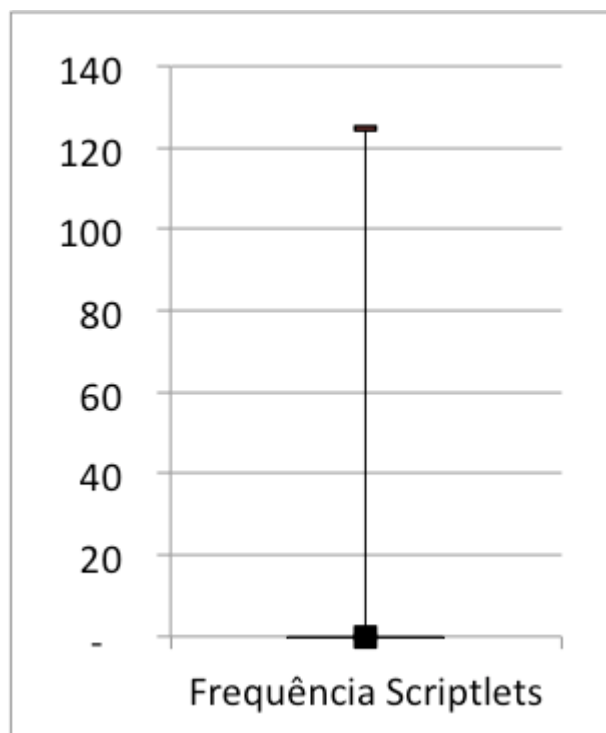
No gráfico é possível perceber uma grande quantidade de arquivos que não contém scriptlets.

Gráfico 1 Histograma de Scriptlet



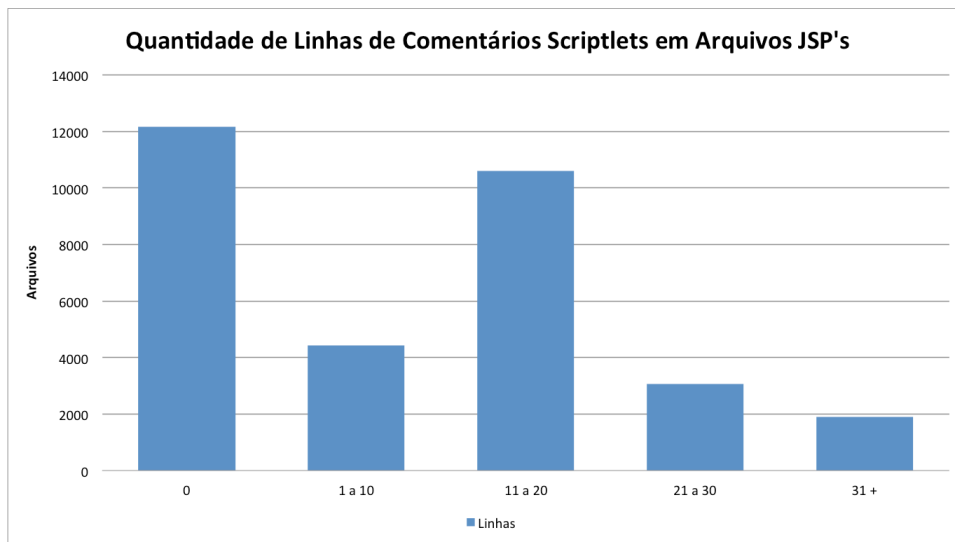
O gráfico de boxplot abaixo apresenta a concentração desses dados na frequência zero.

Gráfico 2 Boxplot de Frequência de Scriptlet



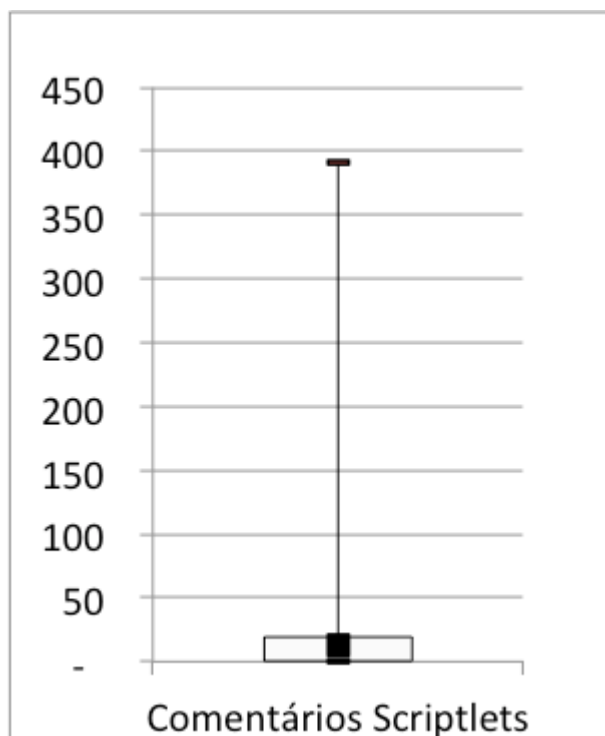
No histograma de comentários de scriptlets fica claro a diminuição de scriptlets, abaixo é possível perceber o aumento de comentários scriptlets em arquivos jsp. Essas informações também nos fez perceber que em média temos de 11 a 20 linhas de comentários scriptlets em arquivos jsp's gerando o que podemos dizer de um possível pagamento da dívida técnica.

Gráfico 3 Histograma Comentários Scriptlets



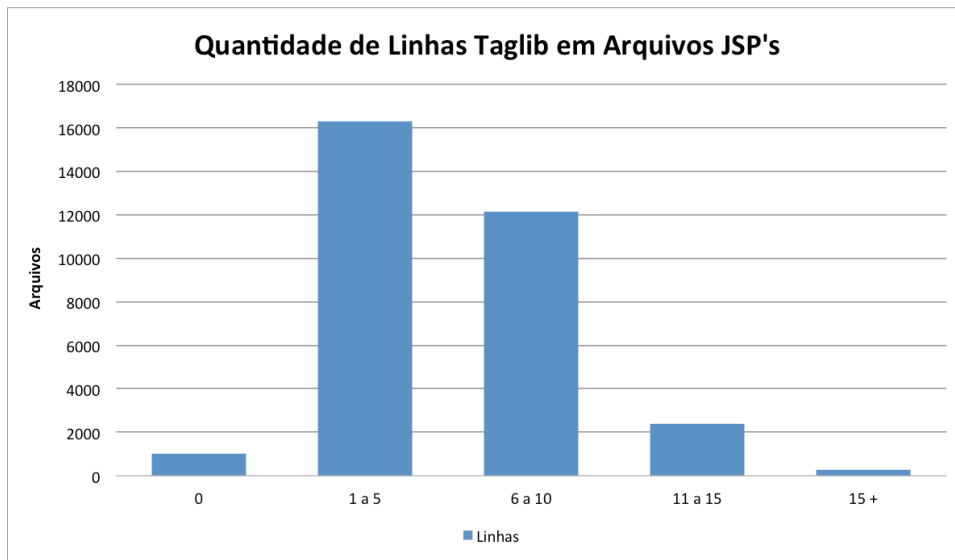
No boxplot é possível visualizar a concentração desses dados que ficam abaixo da frequência 50.

Gráfico 4 Boxplot Frequência Comentários Scriptlets



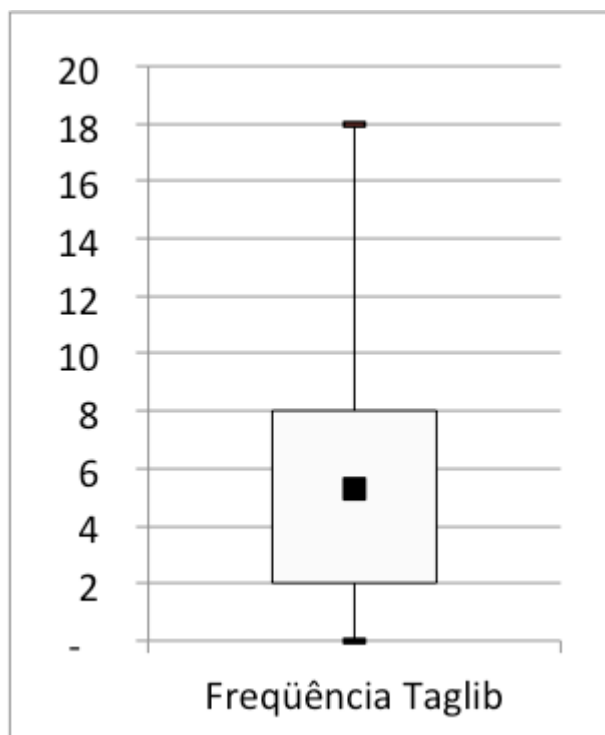
No histograma de taglib é possível perceber que quase todos os arquivos possuem taglibs em seus conteúdos, e que a maior concentração desses dados ocorrem no intervalo de 1 a 10 linhas.

Gráfico 5 Histograma de Taglib



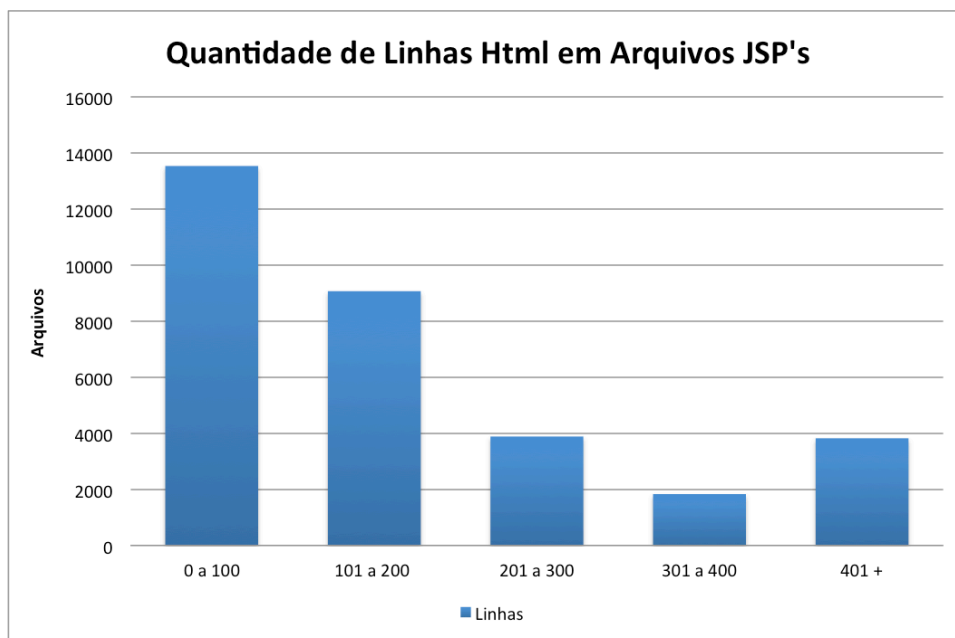
O boxplot de taglib apresenta a concentração desses dados no intervalo de 2 a 8 linhas taglib por arquivo.

Gráfico 6 Bloxplot Frequência Taglib



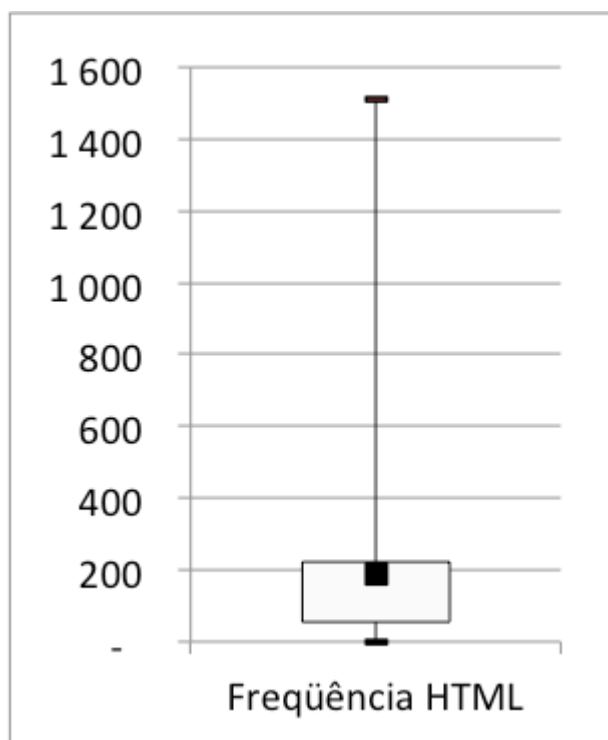
Podemos observar que existe uma frequência alta para arquivos com quantidade de 0 a 200 linhas de HTML. Por isso, foi possível perceber que os arquivos jsp's com conteúdos HTML costumam ser pequenos. Em média temos de 0 a 200 linhas HTML em arquivos.

Gráfico 7 Histograma HTML



O boxplot com a concentração desses dados é possível perceber seu intervalo abaixo de 200 linhas.

Gráfico 8 Boxplot Frequência HTML



5 CONSIDERAÇÕES FINAIS

Com o estudo realizado foi possível perceber que há uma diminuição de scriptlets. A métrica de comentários scriptlets também nos proporcionou uma visão desta diminuição, mostrando um aumento de comentários nesses arquivos.

O aumento de taglibs nos arquivos nos fez perceber que a maioria das empresas estão evitando scriptlets e criando o que chamamos de tags customizadas, isso por um lado diminui a utilização de scriptlets, que é ótimo, mas, por outro, deixa o sistema "preso" a essa customização.

Foi possível também perceber que os arquivos jsps costumam ser pequenos. Arquivos pequenos facilitam a manutenção gerando diminuição de custos para as empresas. Com a diminuição de scriptlets, códigos Java misturados a comandos html, ou seja, camada de Controle sendo manipulada na camada de Visualização, foi possível perceber que mudanças na camada de Visualização não interferem tanto nas outras camadas, porque cada camada tem sua responsabilidade não sendo necessário o conhecimento de outras.

REFERÊNCIAS

- REENSKAUG, Trygve M. H., **Model-View-Controller (MVC)**, Disponível em: <<http://folk.uio.no/trygver/>> [Online; Acesso em: 13/12/2014].
- KRASNER, Glenn E. et al. **A description of the model-view-controller user interface paradigm in the smalltalk-80 system**. Journal of object oriented programming, v. 1, n. 3, p. 26-49, 1988.
- SZOLKOWSKI, Mark. **JavaServer pages: o guia do desenvolvedor**. Elsevier Brasil, 2003.
- NAGAPPAN, Nachiappan; BALL, Thomas; ZELLER, Andreas. Mining metrics to predict component failures. In: **Proceedings of the 28th international conference on Software engineering**. ACM, 2006. p. 452-461.
- LIU, Bing. **Web data mining**. Springer-Verlag Berlin Heidelberg, 2007.
- FAYYAD, Usama; PIATETSKY-SHAPIO, Gregory; SMYTH, Padhraic. From data mining to knowledge discovery in databases. **AI magazine**, v. 17, n. 3, p. 37, 1996.
- Tom Mens and Serge Demeyer. 2007. **Software Evolution**. (Ed.) Manuscript handed over to Publisher on September, 19th 2007, pages 37–67 Acesso em [22-11-2014]. <http://lore.ua.ac.be/Teaching/SRe2LIC/SwEvolutionMerged.pdf>
- J. Li, T. Stålhane, J. M. Kristiansen, and R. Conradi. **Cost drivers of software corrective maintenance: An empirical study in two companies**. In Software Maintenance (ICSM), 2010 IEEE International Conference on, pages 1–8. IEEE, 2010.
- N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya, et al. **Managing technical debt in software-reliant systems**. In Proceedings of the FSE/SDP workshop on Future of software engineering research, pages 47–52. ACM, 2010.
- I. Ozkaya, P. Kruchten, R. L. Nord, and N. Brown. **Managing technical debt in software development**: report on the 2nd international workshop on managing technical debt, held at icse 2011. ACM SIGSOFT Software Engineering Notes, 36(5):33–35, 2011.
- D. Falessi, P. Kruchten, R. L. Nord, and I. Ozkaya. **Technical debt at the crossroads of research and practice**: report on the fifth international workshop on managing technical debt. ACM SIGSOFT Software Engineering Notes, 39(2):31–33, 2014.
- SOKOL, Francisco. **MetricMiner**: uma ferramenta web de apoio à mineração de repositórios de software.
- F. Buschmann. **To pay or not to pay technical debt**. Software, IEEE, 28(6):29–31, 2011.
- BROWN, Nanette et al. **Managing technical debt in software-reliant systems**. In: **Proceedings of the FSE/SDP workshop on Future of software engineering research**. ACM, 2010. p. 47-52.

M. Fowler. **Technicaldebtquadrant.**
<http://www.martinfowler.com/bliki/TechnicalDebtQuadrant.html>, 2009. [Online; Acesso em: 22/11/2014].

ANEXO